

TaskFlow

Diseño e implementación de un pipeline DevOps de extremo a extremo

La aplicación es sencilla a propósito. Lo que construimos no es un gestor de tareas: es el camino automatizado que lleva un commit a producción con confianza.

El problema que resuelve un pipeline

Sin pipeline

- El despliegue es manual y distinto cada vez.
- Los errores se descubren **en producción**.
- Nadie sabe si el servicio está sano hasta que un usuario reclama.
- Volver atrás es improvisar bajo presión.

Con pipeline

- Cada commit atraviesa las **mismas seis puertas**, siempre.
- El fallo aparece en menos de un minuto.
- Métricas, logs y alertas responden por el servicio.
- El rollback es un comando con el SHA del commit.

Arquitectura de la aplicación

```
Navegador (HTML/CSS/JS, sin frameworks)
  | HTTP / JSON
  ▼
Express — middlewares — rutas — controladores — servicio — SQLite
                                     (toda la lógica de negocio)
    |
    ├── métricas Prometheus /metrics
    └── logs JSON con requestId (Winston)
```

- Arquitectura **en capas**: los controladores no ejecutan SQL; el servicio no conoce Express.
- Esa separación permite **31 pruebas unitarias sin levantar un servidor**.
- `app.js` construye la aplicación, `server.js` la arranca: Supertest monta app directamente.

Stack tecnológico

CAPA	HERRAMIENTA	POR QUÉ
Aplicación	Node 20 · Express · SQLite	Cero servicios externos
Pruebas	Jest · Supertest	54 pruebas · 92 % cobertura
Calidad	ESLint 9 · Prettier · npm audit	Bloquea la build
Contenedores	Docker multi-etapa · Compose	Imagen mínima, sin root
CI/CD	GitHub Actions	8 jobs, sin infraestructura propia
Métricas	prom-client · Prometheus	Formato estándar
Logs	Winston · Promtail · Loki	JSON consultable
Alertas	Alertmanager	Agrupación e inhibición

El pipeline



Tres decisiones que defendemos

1 · Validación en paralelo

Feedback de fallo en ~1 min en lugar de ~3. En un pipeline se optimiza el tiempo hasta el **fallo**, no hasta el éxito.

2 · Los PR no publican

Código propuesto no es código aceptado. No debe poder inyectar artefactos en el registro antes de la revisión.

3 · Smoke tests reales

Jest valida el código; no detecta que el Dockerfile olvidó copiar `public/`. Son verificaciones distintas.

Guion de la demostración

#	QUÉ SE MUESTRA	TIEMPO
1	La aplicación funcionando: crear y completar una tarea	45 s
2	<code>npm test</code> — 54 pruebas en 2,5 segundos	45 s
3	Romper la validación a propósito → 5 pruebas fallan → el commit no se despliega	90 s
4	El pipeline en GitHub Actions: paralelismo, fan-in, resumen	60 s
5	Generar tráfico y ver Grafana reaccionar en vivo	60 s
6	<code>docker compose stop app</code> → la alerta pasa a FIRING	60 s
7	Loki: buscar un error por su <code>requestId</code>	30 s

Monitoreo: los tres pilares

Métricas

Prometheus + Grafana

¿Cómo se comporta el sistema?

Logs

Winston + Loki

¿Qué pasó en esta petición?

Alertas

Alertmanager

¿Cuándo debo intervenir?

Detalle que demuestra criterio: cardinalidad

Las rutas se normalizan a `/api/tasks/:id` en lugar de `/api/tasks/42`. Sin eso, mil tareas generarían mil series temporales y Prometheus se degradaría. En Promtail solo `level`, `route` y `statusCode` se promueven a etiquetas de Loki.

Alertas configuradas

ALERTA	CONDICIÓN	ESPERA	SEVERIDAD
AplicacionCaida	up == 0	1 min	critical
TasaDeErroresAlta	> 5 % de 5xx	2 min	critical
ExcesoDeErroresDeCliente	> 5 req/s con 4xx	5 min	warning
LatenciaAlta	p95 > 1 s	3 min	warning
MemoriaElevada	RSS > 400 MB	5 min	warning
ReinicioDelServicio	uptime < 5 min	inmediata	info

La cláusula for, la agrupación y las reglas de **inhibición** son lo que separa un sistema que se consulta de uno que se silencia.

DESAFÍOS

Problemas reales y cómo se resolvieron

DESAFÍO	SOLUCIÓN
Las pruebas se contaminaban entre sí	<code>NODE_ENV=test</code> → <code>SQLite :memory:</code> + <code>reset</code> en cada <code>beforeEach</code>
<code>/api/tasks/stats</code> devolvía 400	Express la capturaba como <code>/:id</code> : declarar la ruta literal antes
Cardinalidad explosiva en Prometheus	Normalizar la etiqueta <code>route</code> con <code>req.route.path</code>
Imagen Docker de más de 1 GB	Build multi-etapa: runtime solo recibe producción
<code>docker stop</code> cortaba peticiones en vuelo	<code>tini</code> como PID 1 + <code>server.close()</code> con <code>timeout</code>
Una caída disparaba seis alertas	Regla de inhibición en Alertmanager

Lecciones aprendidas

- **Automatizar temprano cuesta menos que automatizar tarde.** Montamos el pipeline con la aplicación a medias; cada funcionalidad posterior nació ya validada.
- **La observabilidad se diseña, no se añade.** El `requestId`, la normalización de rutas y la separación `liveness/readiness` son decisiones de la aplicación, no del stack de monitoreo.
- **Los smoke tests cubren un hueco que las unitarias no ven.** Nuestro primer contenedor pasaba `npm test` y devolvía 404 en `/`: el `.dockerignore` excluía `public/`.
- **Una alerta ruidosa es peor que ninguna alerta.** La primera versión disparaba **LatenciaAlta** en cada despliegue.
- **La documentación operativa se escribe pensando en el incidente**, no en la arquitectura.

Cumplimiento del enunciado

54

PRUEBAS

92 %

COBERTURA

8

JOBS CI/CD

6

SERVICIOS DOCKER

6

REGLAS DE ALERTA

5

DOCUMENTOS

Repositorio Git · Pipeline CI/CD · Contenedores · Pruebas unitarias y de integración · Análisis estático · Logs centralizados · Métricas · Alertas · Documentación completa.

Trabajo futuro

- Escaneo de la imagen con **Trivy** dentro del pipeline.
- Pruebas **end-to-end** de navegador con Playwright.
- Migración a **PostgreSQL** para permitir múltiples réplicas.
- **Despliegue canario** con incremento progresivo de tráfico.
- **Trazas distribuidas** con OpenTelemetry: el tercer pilar de la observabilidad.
- **Autenticación** de usuarios y autorización por rol.

Gracias

Preguntas

[Repositorio](#) · [README.md](#) · [docs/INSTALACION.md](#) · [docs/PIPELINE.md](#) · [docs/OPERACIONES.md](#)